

VIPER: The Python IoT Design Suite

PRIYA RAVINDRAN

VIPER, the acronym for Viper Is Python Embedded in Real-Time, is a solution for making your design interactive and connected to the Internet of Things (IoT) easily. The package offers a set of software and hardware tools to seamlessly implement your embedded design ideas, without worrying about actually putting together stuff using solders to make your design work.

The suite is open source and released under general-public licence version 3 (GPL3), Apache2 and Massachusetts Institute of Technology (MIT) licences, depending on the libraries and codes put to use. For commercial usage, their respective websites offer a separate package along with added support. Built to be cross-platform, VIPER is a development suite for high-level design of interactive objects, artistic installations and Internet/cloud-connected devices.

What it offers

VIPER provides a platform for developing embedded designs in Python, using paradigms and features typical of personal computers and mobile programming. It lets you make your object connected and smart with just a few lines of code and almost no wiring.

Multi-board compatibility. Converting your product idea into a prototype can be easily achieved with VIPER, which can run on all 32-bit ARM processors and professional hardware based on ARM microcontrollers (MCUs). It also offers support to do-it-yourself MCU boards like Arduino, UDOO, ST Nucleo, Particle and all Spark products. Creators of VIPER, Things On Internet (TOI), have also come up with their own TOI shield, a multi-sensor board with

The Hello World of Embedded Devices: A Blinking LED Program

```
#####
# Led Blink
# Created by VIPER Team 2015 CC
# Authors: G. Baldi, D. Mazzei
#####

# D0 to D127 represent the names of digital pins
# On most Arduino-like boards pin D13 has an on-board LED connected.
# Initialise a digital pin as an output to drive the connected LED.

pinMode(D13,OUTPUT)

# loop forever
while True:
    digitalWrite(D13,HIGH) # turn the LED ON by making the voltage HIGH
    sleep(1000)            # wait for a second
    digitalWrite(D13,LOW)  # turn the LED OFF by making the voltage LOW
    sleep(1000)            # wait for a second
```

ready-to-plug communication ports. The shield claims to be a first of its kind, enabling plugging on Arduino and Particle boards without any adaptors. To program using VIPER, you just have to make sure you install the universal serial bus (USB) driver corresponding to your board. The board has to 'viperised' before you can use it, and this involves installing VIPER virtual machine (VM) and the required real-time operating system (RTOS) on the board.

A multi-threaded RTOS. Using ChibiOS as the embedded device OS, VIPER permits hardware-architecture-independent design implementation. Implementing multi-threading on top of the OS has brought a new dimension to VIPER. Every VIPER thread is an RTOS thread with a specific priority, and these are synchronised using a global interpreter lock that is released when a higher priority one requests processing.

Elegant handling of peripherals. Abstraction layer of ChibiOS supports basic peripherals like general-purpose input/output (I/O), pulse width modulation (PWM), analogue-to-digital

converters, inter-integrated circuits, serial peripheral interfaces, timers, external interrupts, serials and USBs. The required interface codes can be found as modules in Python library. For other drivers, all that is required is C-function that can be embedded in the script. What this means is that, most Raspberry Pi drivers you work with can be easily adapted and ported into VIPER Python scripts.

Libraries for scripting. Useful modules from the standard Python library, like json, math and urlparse, have been ported to VIPER, and these can be used while working with codes. Along with these, VIPER adds a set of built-ins that can be used directly. You can also import modules while working with sensors, shields or even specific ones pertaining to VIPER threads and connections.

Efficient management. Memory and space management are up to the mark with VIPER. With an intentional effort at minimising object space, built-in functions occupy just four bytes, against 16 for a normal one. It implements a novel memory management system and a memory allocator